

Configuration Proof

1. High Level Conclusion

Branch 1 (the slow path, for real proposals)

```
ProposerSafe (5/11) --module--> Delay Modifier (+ PauseGuard) --module--> OwnerlessSafe --DEVOPS_ROLE--> Term Protocol
```

Branch 2 (the token-vote path, whose only power is to veto)

```
Governor --module--> Roles (+ SetTxNonceGuard) --target--> DelayOwnerSafe --owner--> Delay Modifier
```

Pause controls

```
PauseSafe (2/9) --pause--> PauseGuard
```

```
Admin Safe --unpause / setPauser--> PauseGuard
```

The conclusions drawn from these proofs establish the following generalizations:

1. Only the Term multisig holders on the ProposerSafe and OwnerlessSafe may execute DEVOPS_ROLE methods, holding: 1. the owners, fallbackHandler, modules, guard constant for the ProposerSafe and OwnerlessSafe, and 2. the owner, target, modules, guard, txCooldown, txExpiration constant for the Delay Modifier.
2. Only TERM token holders and Term multisig holders may veto Delay Modifier transactions, holding: 1. the owners, fallbackHandler, modules, guard constant for the DelayOwnerSafe, and 2. the owner, target, modules, guard constant for the Delay Modifier and Roles Modifier, and 3. the role targets, role scopes, role assignments constant for the Roles Modifier, and 4. the voting token, proposal threshold, voting threshold, quorum constant for the Governor.
3. The following: 1. the owners, fallbackHandler, modules, guard for the ProposerSafe, OwnerlessSafe, and DelayOwnerSafe, and 2. the owner, target, modules, guard, txCooldown, txExpiration for the Delay Modifier can be changed only by the Term multisig holders on the ProposerSafe, OwnerlessSafe, or DelayOwnerSafe.
4. The following: 1. the owner, target, modules, guard, role targets, role scopes, role assignments for the Roles Modifier, and 2. the voting token, proposal threshold, voting threshold, quorum for the Governor cannot be changed by anyone.

Only Term multisig holders can execute DEVOPS_ROLE methods subject to TERM token governance, and this cannot be changed other than through Term multisig approval.

2. Evidence Classification

This document uses two classes of evidence to establish the security and functional properties of the governance configuration:

- **Formal Verification (FV) evidence** establishes that, assuming the stated configuration, the relevant contracts enforce the claimed security and functional properties. These properties are proved with the Certora Prover, a formal verification tool. The proofs cover privileged entry points, authorized callers, configuration immutability, the absence of alternative execution paths, and the intended functionality. In the tables below, FV evidence is labeled **FV —** and names each Certora rule;  marks a rule that passes.
- **Deployment evidence** establishes that the deployed contracts have the configuration assumed by the formal verification evidence. This includes on-chain verification of contract addresses, owners, modules, guards, thresholds, roles, and other relevant state, with each value read at a stated block. In the tables below, deployment evidence is labeled **On-chain —**;  marks a read that matches the stated configuration.

These two classes of evidence are complementary. Formal verification evidence does not by itself establish that the deployed contracts have the configuration being proved, and deployment evidence does not by itself establish that the deployed configuration enforces the claimed security and functional properties.

Configuration items marked  (pending migration) describe the intended post-migration configuration and are not established by the current deployment evidence. These items must be re-verified against on-chain state after the migration is executed.

3. Evidence

Generalization 1 — Only the Term multisig holders on the ProposerSafe and OwnerlessSafe may execute DEVOPS_ROLE methods, holding: 1. the owners, fallbackHandler, modules, guard constant for the ProposerSafe and OwnerlessSafe, and 2. the owner, target, modules, guard, txCooldown, txExpiration constant for the Delay Modifier.

#	Statement	Evidence
G1.1	Only the OwnerlessSafe holds the DEVOPS_ROLE required for protocol changes.	On-chain — OwnerlessSafe holds DEVOPS_ROLE on each core contract: ✅ at block 26080131, DEVOPS_ROLE (keccak256("DEVOPS_ROLE") = 0x793a6c9b7e0a9549c74edc2f9ae0dc50903dfaa9a56fb0116b27a8c71de3e2c6) is held by the OwnerlessSafe (0xb8A1dF43c1c88b13937C0c5CEBbAd15830cAeC03) on: • TermController (0xd902EBb8AEb832643af38f43d466a6155cD8Bd5A), read by hasRole • TermEventEmitter (0xf021B31282a60528B2F47D07Ce353da870be78b3), read by hasRole • TermPriceConsumerV3 (0x13Ca4ddB295d621761057D682a1c7b5f5D7Bba4c), read by hasRole • TermRepoDeployerFactory (0x44058C32B154A516F3b9DE2413e6cb938F93c3B4), read by hasRole • TermInitializer (0x3B668F408293e9a5700D6Eae78E1f945CF501eF), read by hasRole • TermDiamond (0xb4B1de03F229220eDbBB31A224b2a004b3c3e47f), read by storage slot 0x5fdec70149aea3337a70dda36f71a4911e25e15ef0e98916a983b6ff4c6aa5a4 = 1 On-chain — DEVOPS_ROLE has only ever been granted to the OwnerlessSafe: ✅ Blockscout's full log history for each contract holds exactly one event naming DEVOPS_ROLE: a RoleGranted to the OwnerlessSafe, emitted in the contract's own creation transaction. There is no RoleRevoked, no RoleAdminChanged, and no grant to any other address. • TermController: block 25188635, tx 0xdeea44228ef14d20a66a02e0a3d84907bcd854aa0f352a5ad8233b48b23f7f15 • TermEventEmitter: block 25188643, tx 0x15c3af7845f086fca8a069ea755f92f4613ae716978aa100089788d98a68d29e • TermPriceConsumerV3: block 25188652, tx 0x7b960b5c31408ac8cf6eb77766c2c8f747bf74b9f2ec1fd98f1d66999a3d6063 • TermRepoDeployerFactory: block 25188657, tx 0x22c72baf2cf6a6ad808e874ca9d75190592059fa9e0591451ea738147394f8f5 • TermInitializer: block 25555701, tx 0xec1c9e2935569e769123e7a05f6d1f5fc2c1ba7aa2a7b4013d799dfee76885d7 • TermDiamond: block 25188499, tx 0xe70a739ab7ff78a6b8ae8beefd002c7ee4a87d8c41b6ec594ae45eba87412788 On-chain — Nobody can grant additional DEVOPS_ROLE holders with grantRole:

#	Statement	Evidence
		•  on the five AccessControl contracts, <code>getRoleAdmin(DEVOPS_ROLE)</code> is <code>DEFAULT_ADMIN_ROLE (0x00)</code>. No production contract grants <code>DEFAULT_ADMIN_ROLE</code> or calls <code>_setRoleAdmin</code>, and no <code>RoleGranted</code> for <code>DEFAULT_ADMIN_ROLE</code> appears in any of the six histories, so no address can call <code>grantRole(DEVOPS_ROLE, ...)</code>. • On the <code>TermDiamond</code>, <code>grantRole</code> is not registered at all: its roles are set once by <code>initDiamond</code>, or by a new facet added through <code>diamondCut</code>, which is itself <code>DEVOPS_ROLE</code>-only.
G1.2	Only Modules or Owners can interact with <code>OwnerlessSafe</code> , except <code>fallback</code> and <code>receive</code> , which anyone can call.	FV — Every entry point to the Safe is enumerated below: SE-1 <code>safeEntryPointsAreAllAccountedFor</code>  passes: <code>GnosisSafe v1.3.0</code> has no write functions besides these seventeen entry points (the Prover checks <code>fallback</code> and <code>receive</code> as one entry), and any unsupported call lands in <code>fallback</code>. Entry points are grouped below by who can call each one: <i>Module-gated entry points:</i> • <code>execTransactionFromModule</code> • <code>execTransactionFromModuleReturnData</code> <i>Owner-gated entry points:</i> • <code>execTransaction</code> • <code>approveHash</code> <i>Self calls — only modules and owners can reach these:</i> • <code>enableModule</code> • <code>disableModule</code> • <code>addOwnerWithThreshold</code> • <code>removeOwner</code> • <code>swapOwner</code> • <code>changeThreshold</code> • <code>setGuard</code> • <code>setFallbackHandler</code> <i>Non-interactions — these always revert, so none of their effects persist:</i> • <code>setup</code> • <code>requiredTxGas</code> • <code>simulateAndRevert</code> <i>Open to anyone:</i> • <code>fallback</code> • <code>receive</code> FV — Module-only entry points can make the Safe execute to any destination: <code>execTransactionFromModule</code>, <code>execTransactionFromModuleReturnData</code>. • SE-3 <code>onlyEnabledModulesCanCallModuleExec</code>  passes: only an enabled module can successfully call <code>execTransactionFromModule</code> or <code>execTransactionFromModuleReturnData</code>. • SE-4 <code>enabledModuleCanMakeTheSafeAct</code>  passes: an enabled module can call <code>execTransactionFromModule</code> and <code>execTransactionFromModuleReturnData</code> directly, with no owner signatures, and the Safe makes an outgoing call or delegatecall. FV — Owner-only entry points can make the Safe execute to any destination: <code>execTransaction</code>, <code>approveHash</code>. • SE-5 <code>execTransactionRunsOnlyAfterTheSignatureCheck</code>  passes: <code>execTransaction</code> only runs once the signature threshold check has passed for that exact transaction.

#	Statement	Evidence
		• SE-12 <code>eachSignatureAcceptsANewApprovingOwner</code> ✓ passes: each accepted signature comes from a different owner who approved it. With SE-5, the Safe acts through <code>execTransaction</code> only when as many different owners as the threshold have approved. • SE-8 <code>onlyOwnersCanApproveHashes</code> ✓ passes: <code>approveHash</code> only records an owner's own approval, which then counts as that owner's signature in <code>execTransaction</code>. • SE-6 <code>ownersCanMakeTheSafeAct</code> ✓ passes: once the threshold is passed, <code>execTransaction</code> succeeds and the Safe acts, for any threshold and any number of owners. FV — Settings functions are only reached through the module-only or owner-only points: • SE-7 <code>settingsOnlyChangeWhenTheSafeCallsItself</code> ✓ passes: these functions only change a setting when the caller is the Safe itself. • SC-4 <code>settingsOnlyChangeThroughAModuleOrOwners</code> ✓ passes: over every write function, <code>fallback</code> included, a setting only changes through an enabled module's <code>execTransactionFromModule</code> or <code>execTransactionFromModuleReturnData</code>, or through <code>execTransaction</code>. A fallback handler is treated as the Safe itself, so it cannot reach these functions even then. • SC-1 <code>moduleCanChangeSettings</code>, SC-2 <code>moduleCanChangeSettingsWithReturnData</code> ✓ pass: for each function above, a module's <code>execTransactionFromModule</code>, and likewise its <code>execTransactionFromModuleReturnData</code>, addressed to the Safe with a call to that function, succeeds and changes a setting. • SC-3 <code>ownersCanChangeSettings</code> ✓ passes: for each function above, an owner's <code>execTransaction</code>, addressed to the Safe with a call to that function, succeeds and changes a setting, once the threshold is passed. FV — setup is not a callable function after deploy: • SE-9 <code>setupAlwaysRevertsAfterSetup</code> ✓ passes: once the Safe is set up, <code>setup</code> always reverts, so nobody can re-run it to replace the owners, threshold, modules or handler. FV — <code>requiredTxGas</code> and <code>simulateAndRevert</code> always revert: • SE-10 <code>requiredTxGasAlwaysReverts</code>, SE-11 <code>simulateAndRevertAlwaysReverts</code> ✓ pass: anyone can call them, and they do execute a call (<code>requiredTxGas</code>) or <code>delegatecall</code> (<code>simulateAndRevert</code>), but they always revert afterwards, so none of their effects persist. FV — Any address can call any unsupported Safe function to initiate <code>fallback</code>, which can only make the Safe call its fallback handler, and only when one is set to a nonzero address: • SE-14 <code>fallbackOnlyCallsItsHandler</code> ✓ passes: with a handler set, <code>fallback</code> only makes a plain call, with no ETH, to the handler. It never <code>delegatecalls</code>, so the handler's code executes in the handler's own context, not the Safe's context, and cannot write the Safe's storage. Assumes the handler is not the Safe itself: v1.3.0 allows that (v1.4.0 forbids it, GS400). • SE-15 <code>fallbackHandlerCallsComeFromTheHandler</code> ✓ passes: once <code>fallback</code> calls its handler, every onward call the handler makes has <code>msg.sender</code> equal to the handler, not the Safe. Assumes the handler is not the Safe itself; that case is SE-16. • SE-16 <code>selfHandlerFallbackChangesNothing</code> ✓ passes: with the Safe set as its own handler, <code>fallback</code> leaves the Safe's storage exactly as it was, and when it succeeds, the Safe calls nothing but itself, sends no ETH and makes no <code>delegatecall</code>.

#	Statement	Evidence
		- SE-13 <code>fallbackMakesNoCallWithoutAHandler</code> passes: with no handler set (the zero address), <code>fallback</code> makes no call or <code>delegatecall</code> at all. FV — Any address can trigger <code>receive</code> by sending ETH without calldata and it does nothing but emit <code>SafeReceived</code>: - SE-17 <code>receiveOnlyEmitsSafeReceived</code> passes: whenever <code>receive</code> runs, it does nothing beyond the event. It emits exactly one log, <code>SafeReceived</code>, with the caller as the sender. It makes no call of any kind (call, <code>delegatecall</code>, <code>staticcall</code> or <code>callcode</code>) and writes no storage. - SE-18 <code>receiveCanRun</code> passes: some call to the Safe succeeds and emits <code>SafeReceived</code>, so SE-17 is not vacuous.
G1.3	OwnerlessSafe fallback handler is <code>address(0)</code> .	On-chain — OwnerlessSafe fallback handler is <code>address(0)</code>: the fallback handler slot (<code>0x6c9a6c4a39284e37ed1cf53d337577d14212a4870fb976a4366c693b939918d5</code>, <code>FallbackManager.sol:12</code>) on the OwnerlessSafe (<code>0xb8A1dF43c1c88b13937C0c5CEBbAd15830cAeC03</code>) is <code>address(0)</code> at block 26092959.
G1.4	OwnerlessSafe owners are Term multisig holders.	On-chain — Nine keys, all nine needed: at block 26084853, the OwnerlessSafe (<code>0xb8A1dF43c1c88b13937C0c5CEBbAd15830cAeC03</code>) runs GnosisSafe v1.3.0 (VERSION), has threshold 9 (<code>getThreshold</code>) and these 9 owners (<code>getOwners</code>), each an externally owned account with no contract code: <code>0xc1047a4D9f6071B55585523F7F659F39A1fBA74b</code> <code>0x8EF485fD38b7B29a827938C9F00300eaBf8E1710</code> <code>0xbffFcAdCd5549cC378693108Bcd4435776A6fa795</code> <code>0xB680373c50E9E877DA7dCF9efcc0dFB234452ad3</code> <code>0x6eb0c274EC4B1d51152e45BAED22f883B2A3Bc60</code> <code>0xE82183cfAE1C24f044315c318FD97bE7e47b31D2</code> <code>0xeeee661edcFE634Dc0e29D62C26AfD62c0843b817</code> <code>0xDB97c377F23a9Dc54c3D6F9df7bC0ca4Cf7D5A81</code> <code>0x89562FC5AEF155481aE0C7dde1300110B4dF2F1D</code> - The identities of the nine key holders are not visible on chain. Their status as Term multisig holders is a deployment fact.
G1.5	Delay Modifier is the only enabled module on OwnerlessSafe.	On-chain — The OwnerlessSafe's module list contains only the Delay Modifier: - The OwnerlessSafe's (<code>0xb8A1dF43c1c88b13937C0c5CEBbAd15830cAeC03</code>) module list returns just one, the Delay Modifier (<code>0x0C19d8A404079d71E5CA3e32fE3f758Ab543ACdf</code>), and the list ends there (<code>getModulesPaginated</code>, block 26092962). On-chain — The OwnerlessSafe's storage holds only the Delay Modifier as a module: - Read directly from storage rather than through the Safe's own functions, the module list starts at the Delay Modifier and the Delay Modifier points back to the start, so it is the only entry (<code>modules</code>, storage slot 1, block 26092962).
G1.6	Only Modules or Owner can interact with Delay Modifier, except <code>executeNextTx</code> and <code>skipExpired</code> , which anyone can call.	FV — Every entry point to the Delay Modifier is enumerated below: DM-13 <code>delayWriteFunctionsAreTheKnownFifteen</code> passes: the Delay Modifier has no fallback and exactly fifteen write functions, each covered below: <code>setTxCooldown</code> — owner only (DM-8) <code>setTxExpiration</code> — owner only (DM-8) <code>setTxNonce</code> — owner only (DM-8) <code>setAvatar</code> — owner only (DM-8)

#	Statement	Evidence
		• <code>setTarget</code> — owner only (DM-8) • <code>enableModule</code> — owner only (DM-8) • <code>disableModule</code> — owner only (DM-8) • <code>setGuard</code> — owner only (DM-8) • <code>transferOwnership</code> — owner only (DM-8) • <code>renounceOwnership</code> — owner only (DM-8) • <code>execTransactionFromModule</code> — modules only (DM-4) • <code>execTransactionFromModuleReturnData</code> — modules only (DM-4) • <code>executeNextTx</code> — anyone (DM-11) • <code>skipExpired</code> — anyone (DM-12) • <code>setUp</code> — nobody after deployment (DM-1) FV — Owner-only settings: • DM-8 <code>atMostOneCallerPassesOnlyOwner</code>  passes: only the owner can call the Delay Modifier's ten settings functions. There can only be one owner at a time. FV — Module-only queueing: • DM-4 <code>queueOnlyGrowsThroughEnabledModules</code>  passes: only an enabled module can add a transaction to the queue. FV — Setup is closed: • DM-1 <code>setUpAlwaysRevertsAfterDeployment</code>  passes: nobody can re-run the Delay Modifier's setup after deployment. FV — The two exceptions are open: • DM-11 <code>anyoneCanExecuteNextTx</code>, DM-12 <code>anyoneCanSkipExpired</code>  pass: an address that is neither the owner nor a module can execute a queued transaction and skip an expired one. FV — Every other function, checked all at once: • DM-14 <code>onlyModulesOrOwnerCanCallDelay</code>  passes: for every write function except <code>executeNextTx</code>, <code>skipExpired</code> and <code>setUp</code>, a call that succeeds came from the owner or an enabled module. It does not rely on a list of names, so it also covers any function added later.
G1.7	Only enabled modules can queue transactions to Delay Modifier. Therefore, Delay Modifier's owner, <code>DelayOwnerSafe</code> , cannot queue transactions to Delay Modifier.	FV — Only an enabled module can queue: • DM-4 <code>queueOnlyGrowsThroughEnabledModules</code>  passes: over every write function of the Delay Modifier except <code>setUp</code> (closed by DM-1), the queue nonce and every queue entry only change when the caller is an enabled module. The owner has no path to the queue: none of the owner's ten settings functions (DM-8) adds to the queue. On-chain — The owner is not an enabled module: •  at block 26084775, <code>isModuleEnabled</code> on the Delay Modifier (<code>0x0C19d8A404079d71E5CA3e32fE3f758Ab543ACdf</code>) returns false for its current owner, the old Roles (<code>0x405b47354CF06A25DE1DDb35EC65F03939E2e8D2</code>), and for its planned owner, the <code>DelayOwnerSafe</code> (<code>0x2a875746D0c88EBD2bbBfc8F8a773c58c3373ad3</code>). Its only module is the <code>ProposerSafe</code> (G1.8).
G1.8	<code>ProposerSafe</code> is the only enabled module on the Delay Modifier.	On-chain — The Delay Modifier's module list contains only the <code>ProposerSafe</code>: •  The Delay Modifier's (<code>0x0C19d8A404079d71E5CA3e32fE3f758Ab543ACdf</code>) module list returns just one, the <code>ProposerSafe</code> (<code>0xd5E12854A3Db99deF295A7635D3Ba16427d2A28</code>), and reports that the list ends there (<code>getModulesPaginated</code>, block

#	Statement	Evidence
		26080101). On-chain — The Delay Modifier's storage holds only the ProposerSafe as a module: •  Read directly from storage rather than through the Delay Modifier's own functions, the module list starts at the ProposerSafe and the ProposerSafe points back to the start, so it is the only entry (modules, storage slot 104, block 26080101).
G1.9	Only Modules or Owners can interact with ProposerSafe, except fallback and receive, which anyone can call.	Same as G1.2: the ProposerSafe runs the same GnosisSafe v1.3.0 code as the OwnerlessSafe.
G1.10	ProposerSafe fallback handler is address(0).	On-chain — ProposerSafe fallback handler is address(0): •  the fallback handler slot (0x6c9a6c4a39284e37ed1cf53d337577d14212a4870fb976a4366c693b939918d5, FallbackManager.sol:12) on the ProposerSafe (0xd5E12854A3DbA99deF295A7635D3Ba16427d2A28) is address(0) at block 26056133.
G1.11	ProposerSafe owners are Term multisig holders.	On-chain — Eleven keys, any five act:  at block 26084863, the ProposerSafe (0xd5E12854A3DbA99deF295A7635D3Ba16427d2A28) runs GnosisSafe v1.3.0 (VERSION), has threshold 5 (getThreshold) and these 11 owners (getOwners), each an externally owned account with no contract code: • 0xF6A745f9B38FFcd17Ee8909AC89151D788F95282 • 0xC43d527E3544A3d199Ce28E87994384D89d90e6E • 0x8EF485fD38b7B29a827938C9F00300eaBf8E1710 • 0xbfFcAdCd5549cC378693108Bcd4435776A6fa795 • 0xB680373c50E9E877DA7dCF9efcc0dFB234452ad3 • 0x6eb0c274EC4B1d51152e45BAED22f883B2A3Bc60 • 0xE82183cfAE1C24f044315c318FD97bE7e47b31D2 • 0xeeee661edcFE634Dc0e29D62C26AfD62c0843b817 • 0xc1047a4D9f6071B55585523F7F659F39A1fBA74b • 0xDB97c377F23a9Dc54c3D6F9df7bC0ca4Cf7D5A81 • 0x89562FC5AEF155481aE0C7dde1300110B4dF2F1D • The identities of the eleven key holders are not visible on chain. Their status as Term multisig holders is a deployment fact.
G1.12	ProposerSafe has no enabled modules.	On-chain — ProposerSafe has no enabled modules: •  getModulesPaginated on the ProposerSafe (0xd5E12854A3DbA99deF295A7635D3Ba16427d2A28) returns an empty list at block 26050570.
G1.13	ProposerSafe can propose a transaction to the Delay Modifier that is executed on the OwnerlessSafe once the cooldown has passed, the expiration has not passed, the PauseGuard on the Delay Modifier is not paused, and the transaction has not been vetoed.	FV — The ProposerSafe can propose to the Delay Modifier as a module: • DM-5 enabledModuleCanQueue  passes: any enabled module can queue a transaction on the Delay Modifier: its call to execTransactionFromModule, with any destination, value, calldata and operation, succeeds and adds the transaction to the queue. The ProposerSafe is the Delay Modifier's only enabled module (G1.8). FV — A transaction proposed by a module executes after cooldown and before expiration: • DP-29 queuedTransactionExecutesAfterCooldown  passes: if a module has queued a transaction, the cooldown has passed, it has not expired, and the PauseGuard is not paused, anyone can execute it and it reaches the OwnerlessSafe.

#	Statement	Evidence
		FV — A transaction cannot execute too early or too late: - DP-30 <code>executeNextTxRevertsDuringCooldown</code>, DP-31 <code>executeNextTxRevertsAfterExpiration</code>  pass: a queued transaction cannot execute before the cooldown has passed or after it has expired. FV — A transaction cannot be executed while the PauseGuard is paused: - DP-18 <code>pausedBlocksExecuteNextTx</code>  passes: a queued transaction cannot execute while the PauseGuard is paused. FV — Only the transaction at the current <code>txNonce</code> executes: - DP-28 <code>executeNextTxConsumesOnlyTheEntryAtTxNonce</code>  passes: <code>executeNextTx</code> runs exactly the transaction that was queued at the exact <code>txNonce</code>. FV — A transaction cannot be executed once vetoed: - DP-32 <code>vetoedTransactionCannotExecute</code>, DP-33 <code>entryPassedByTxNonceNeverExecutes</code>  pass: a proposal is vetoed by moving <code>txNonce</code> past the proposal's nonce with <code>setTxNonce</code>. Once <code>txNonce</code> has moved past a proposal's nonce, it can never be executed.

Generalization 2 — Only TERM token holders and Term multisig holders may veto Delay Modifier transactions, holding: 1. the owners, fallbackHandler, modules, guard constant for the DelayOwnerSafe, and 2. the owner, target, modules, guard constant for the Delay Modifier and Roles Modifier, and 3. the role targets, role scopes, role assignments constant for the Roles Modifier, and 4. the voting token, proposal threshold, voting threshold, quorum constant for the Governor.

#	Statement	Evidence
G2.1	Veto power (<code>Delay.setTxNonce(uint256)</code>) is held exclusively by the owner of the Delay Modifier.	FV — <code>setTxNonce</code> is owner-only: DM-8 <code>atMostOneCallerPassesOnlyOwner</code>  passes: <code>setTxNonce</code> is one of the Delay Modifier's ten <code>onlyOwner</code> functions: only the address recorded as the owner can call it successfully, and no two different callers can both successfully call it from the same state.
G2.2	Only Modules or Owner can interact with Delay Modifier, except <code>executeNextTx</code> and <code>skipExpired</code> , which anyone can call.	Same as G1.6.
G2.3	Delay Modifier only has one owner, which is the DelayOwnerSafe.	FV — Delay Modifier has one owner: - DM-8 <code>atMostOneCallerPassesOnlyOwner</code>  passes: the Delay Modifier has only one owner: at any moment, only one address can change the Delay Modifier's settings, and it is the address recorded as the owner. No second address can ever do it alongside it. On-chain — Owner is DelayOwnerSafe:  pending migration. <code>owner()</code> on the Delay Modifier must return <code>0x2a875746D0c88EBD2bbBfc8F8a773c58c3373ad3</code>. At block 26049693 it returns <code>0x405b47354CF06A25DE1DDb35EC65F03939E2e8D2</code> (old Roles).
G2.4	Only Modules or Owners can interact with DelayOwnerSafe, except <code>fallback</code> and <code>receive</code> , which anyone can call.	- DelayOwnerSafe uses a different GnosisSafe mastercopy (v1.4.1) than OwnerlessSafe and ProposerSafe (v1.3.0). Evidence uses the same rules as G1.2, run against GnosisSafe v1.4.1. FV — Every entry point to the Safe is enumerated below: SE141-1 <code>safeEntryPointsAreAllAccountedFor</code>  passes: GnosisSafe v1.4.1 has no write functions besides these sixteen entry points (the Prover checks <code>fallback</code> and <code>receive</code> as one entry), and any unsupported call lands in <code>fallback</code> . These are the seventeen entry points in G1.2 minus <code>requiredTxGas</code> , which Safe removed in v1.4.0. Entry points are grouped below by who can call each one: <i>Module-gated entry points:</i> <code>execTransactionFromModule</code> <code>execTransactionFromModuleReturnData</code> <i>Owner-gated entry points:</i> <code>execTransaction</code> <code>approveHash</code> <i>Self calls — only modules and owners can reach these:</i> <code>enableModule</code> <code>disableModule</code> <code>addOwnerWithThreshold</code> <code>removeOwner</code> <code>swapOwner</code> <code>changeThreshold</code>

#	Statement	Evidence
		• <code>setGuard</code> • <code>setFallbackHandler</code> <i>Non-interactions — these always revert, so none of their effects persist:</i> • <code>setup</code> • <code>simulateAndRevert</code> <i>Open to anyone:</i> • <code>fallback</code> • <code>receive</code> FV — Module-only entry points can make the Safe execute to any destination: <code>execTransactionFromModule</code>, <code>execTransactionFromModuleReturnData</code>. • SE141-3 <code>onlyEnabledModulesCanCallModuleExec</code> ✓ passes: only an enabled module can successfully call <code>execTransactionFromModule</code> or <code>execTransactionFromModuleReturnData</code>. • SE141-4 <code>enabledModuleCanMakeTheSafeAct</code> ✓ passes: an enabled module can call <code>execTransactionFromModule</code> and <code>execTransactionFromModuleReturnData</code> directly, with no owner signatures, and the Safe makes an outgoing call or <code>delegatecall</code>. FV — Owner-only entry points can make the Safe execute to any destination: <code>execTransaction</code>, <code>approveHash</code>. • SE141-5 <code>execTransactionRunsOnlyAfterTheSignatureCheck</code> ✓ passes: <code>execTransaction</code> only runs once the signature threshold check has passed for that exact transaction. • SE141-12 <code>eachSignatureAcceptsANewApprovingOwner</code> ✓ passes: each accepted signature comes from a different owner who approved it. With SE141-5, the Safe acts through <code>execTransaction</code> only when as many different owners as the threshold have approved. • SE141-8 <code>onlyOwnersCanApproveHashes</code> ✓ passes: <code>approveHash</code> only records an owner's own approval, which then counts as that owner's signature in <code>execTransaction</code>. • SE141-6 <code>ownersCanMakeTheSafeAct</code> ✓ passes: once the signature threshold is passed, <code>execTransaction</code> succeeds and the Safe acts, for any threshold and any number of owners. FV — Settings functions are only reached through the module-only or owner-only points: • SE141-7 <code>settingsOnlyChangeWhenTheSafeCallsItself</code> ✓ passes: these functions only change a setting when the caller is the Safe itself. • SC141-4 <code>settingsOnlyChangeThroughAModuleOrOwners</code> ✓ passes: over every write function, <code>fallback</code> included, a setting only changes through an enabled module's <code>execTransactionFromModule</code> or <code>execTransactionFromModuleReturnData</code>, or through <code>execTransaction</code>. A fallback handler is treated as the Safe itself, so it cannot reach these functions even then. • SC141-1 <code>moduleCanChangeSettings</code>, SC141-2 <code>moduleCanChangeSettingsWithReturnData</code> ✓ pass: for each function above, a module's <code>execTransactionFromModule</code>, and likewise its <code>execTransactionFromModuleReturnData</code>, addressed to the Safe with a call to that function, succeeds and changes a setting. • SC141-3 <code>ownersCanChangeSettings</code> ✓ passes: for each function above, an owner's <code>execTransaction</code>, addressed to the Safe with a call to that function, succeeds and changes a setting once the threshold is passed. FV — <code>setup</code> is not a callable function after deploy:

#	Statement	Evidence
		• SE141-9 <code>setupAlwaysRevertsAfterSetup</code>  passes: once the Safe is set up, <code>setup</code> always reverts, so nobody can re-run it to replace the owners, threshold, modules or handler. FV — <code>simulateAndRevert</code> always reverts: • SE141-11 <code>simulateAndRevertAlwaysReverts</code>  passes: anyone can call <code>simulateAndRevert</code>, and it does make a <code>delegatecall</code>, but it always reverts afterwards, so none of its effects persist. FV — Any address can call any unsupported Safe function to initiate fallback, which can only make the Safe call its fallback handler, and only when one is set to a nonzero address: • SE141-14 <code>fallbackOnlyCallsItsHandler</code>  passes: with a handler set, <code>fallback</code> only makes a plain call, with no ETH, to the handler. It never <code>delegatecalls</code>, so the handler's code executes in the handler's own context, not the Safe's context, and cannot write the Safe's storage. Assumes the handler is not the Safe itself: v1.4.1's <code>setFallbackHandler</code> and <code>setup</code> refuse that (GS400), but a <code>delegatecall</code> run through <code>execTransaction</code> or a module can still write the handler slot directly. That case is covered by SE141-16. • SE141-15 <code>fallbackHandlerCallsComeFromTheHandler</code>  passes: once <code>fallback</code> calls its handler, every onward call the handler makes has <code>msg.sender</code> equal to the handler, not the Safe. Assumes the handler is not the Safe itself; that case is SE141-16. • SE141-16 <code>selfHandlerFallbackChangesNothing</code>  passes: with the Safe set as its own handler, <code>fallback</code> leaves the Safe's storage exactly as it was, and when it succeeds, the Safe calls nothing but itself, sends no ETH and makes no <code>delegatecall</code>. • SE141-13 <code>fallbackMakesNoCallWithoutAHandler</code>  passes: with no handler set (the zero address), <code>fallback</code> makes no call or <code>delegatecall</code> at all. FV — Any address can trigger <code>receive</code> by sending ETH without calldata and it does nothing but emit <code>SafeReceived</code>: • SE141-17 <code>receiveOnlyEmitsSafeReceived</code>  passes: whenever <code>receive</code> runs, it does nothing beyond the event. It emits exactly one log, <code>SafeReceived</code>, with the caller as the sender. It makes no call of any kind (call, <code>delegatecall</code>, <code>staticcall</code> or <code>callcode</code>) and writes no storage. • SE141-18 <code>receiveCanRun</code>  passes: a call to the Safe with no calldata and ETH attached succeeds and emits <code>SafeReceived</code>, so SE141-17 is not vacuous.
G2.5	DelayOwnerSafe fallback handler is <code>address(0)</code> .	On-chain — DelayOwnerSafe fallback handler is <code>address(0)</code>: •  the fallback handler slot (<code>0x6c9a6c4a39284e37ed1cf53d337577d14212a4870fb976a4366c693b939918d5</code>, <code>FallbackManager.sol:14</code> in GnosisSafe v1.4.1) on the DelayOwnerSafe (<code>0x2a875746D0c88EBD2bbBfc8F8a773c58c3373ad3</code>) is <code>address(0)</code> at block 26080124.
G2.6	DelayOwnerSafe owners are Term multisig holders.	On-chain — Eleven keys, any five act:  at block 26084873, the DelayOwnerSafe (<code>0x2a875746D0c88EBD2bbBfc8F8a773c58c3373ad3</code>) runs GnosisSafe v1.4.1 (VERSION), has threshold 5 (<code>getThreshold</code>) and these 11 owners (<code>getOwners</code>), each an externally owned account with no contract code: • <code>0xeeee661edcFE634Dc0e29D62C26AfD62c0843b817</code> • <code>0xF6A745f9B38FFcd17Ee8909AC89151D788F95282</code> • <code>0xC43d527E3544A3d199Ce28E87994384D89d90e6E</code> • <code>0x8EF485fD38b7B29a827938C9F00300eaBf8E1710</code> • <code>0xbFfcAdCd5549cC378693108Bcd4435776A6fa795</code>

#	Statement	Evidence
		• <code>0xB680373c50E9E877DA7dCF9efcc0dFB234452ad3</code> • <code>0x6eb0c274EC4B1d51152e45BAED22f883B2A3Bc60</code> • <code>0xE82183cfAE1C24f044315c318FD97bE7e47b31D2</code> • <code>0xc1047a4D9f6071B55585523F7F659F39A1fBA74b</code> • <code>0xDB97c377F23a9Dc54c3D6F9df7bC0ca4Cf7D5A81</code> • <code>0x89562FC5AEF155481aE0C7dde1300110B4dF2F1D</code> • The identities of the eleven key holders are not visible on chain. Their status as Term multisig holders is a deployment fact.
G2.7	DelayOwnerSafe only has one enabled module, the Roles Modifier.	On-chain — The DelayOwnerSafe's module list contains only the Roles Modifier: •  The DelayOwnerSafe's (<code>0x2a875746D0c88EBD2bbBfc8F8a773c58c3373ad3</code>) module list returns just one, the Roles Modifier (<code>0xaBAC51B6AEb05a2CE65310F79e64DF203D6c8Ab3</code>), and the list ends there (<code>getModulesPaginated</code>, block 26080124). On-chain — The DelayOwnerSafe's storage holds only the Roles Modifier as a module: •  Read directly from storage rather than through the Safe's own functions, the module list starts at the Roles Modifier and the Roles Modifier points back to the start, so it is the only entry (<code>modules</code>, storage slot 1, block 26080124).
G2.8	Only Modules with assignedRoles or Owner can interact with Roles Modifier.	FV — Every entry point to the Roles Modifier is enumerated below: RI-14 <code>rolesWriteFunctionsAreTheKnownTwentyFive</code>  passes: the Roles Modifier has no <code>fallback</code> or <code>receive</code> and no write functions besides these twenty-five. Entry points are grouped below by who can call each one: <i>Entry points gated to enabled modules with assigned roles:</i> • <code>execTransactionFromModule</code> (the caller's default role) • <code>execTransactionFromModuleReturnData</code> (the caller's default role) • <code>execTransactionWithRole</code> (the role the call names) • <code>execTransactionWithRoleReturnData</code> (the role the call names) <i>Owner-gated entry points — the settings functions:</i> • <code>setMultisend</code> • <code>allowTarget</code> • <code>revokeTarget</code> • <code>scopeTarget</code> • <code>scopeAllowFunction</code> • <code>scopeRevokeFunction</code> • <code>scopeFunction</code> • <code>scopeFunctionExecutionOptions</code> • <code>scopeParameter</code> • <code>scopeParameterAsOneOf</code> • <code>unscopeParameter</code> • <code>assignRoles</code> • <code>setDefaultRole</code> • <code>setAvatar</code> • <code>setTarget</code> • <code>enableModule</code> • <code>disableModule</code> • <code>setGuard</code> • <code>transferOwnership</code> • <code>renounceOwnership</code> <i>Non-interactions — this always reverts, so none of its effects persist:</i>

#	Statement	Evidence
		• <code>setUp</code> FV — Execution functions succeed only if the caller address is an enabled module and has been assigned the permissioned role: <code>execTransactionFromModule</code>, <code>execTransactionFromModuleReturnData</code>, <code>execTransactionWithRole</code>, <code>execTransactionWithRoleReturnData</code>. • RI-7 <code>onlyEnabledModulesCanExec</code> ✓ passes: only enabled modules can call exec functions. • RI-8 <code>moduleWithoutDefaultRoleCannotExecFromModule</code> ✓ passes: an enabled module that is not a member of its default role cannot execute through <code>execTransactionFromModule</code> or <code>execTransactionFromModuleReturnData</code>. • RI-9 <code>moduleWithoutRoleCannotExecTransactionWithRole</code>, RI-10 <code>moduleWithoutRoleCannotExecTransactionWithRoleReturnData</code> ✓ pass: an enabled module that is not a member of the role the call names cannot execute through <code>execTransactionWithRole</code> or <code>execTransactionWithRoleReturnData</code>. With RI-8, a module with no assigned role can execute nothing through any of the four. • RI-11 <code>roleMemberModuleCanExecFromModule</code>, RI-12 <code>roleMemberModuleCanExecTransactionWithRole</code>, RI-13 <code>roleMemberModuleCanExecTransactionWithRoleReturnData</code> ✓ pass: an enabled module that is a member of the role the call runs under can successfully call each of the four, so the restriction is not achieved by nothing working. FV — Owner-only settings: • RI-2 <code>rolesConfigOnlyChangesThroughOwner</code>, RI-3 <code>ownerOnlyChangesThroughOwnableTransfer</code>, RI-4 <code>ownerCanStillReconfigure</code>, RI-5 <code>onlyOwnerCanCallRolesSettings</code>, RI-6 <code>ownerCanCallEachRolesSetting</code> ✓ pass: each of the twenty settings functions only succeeds for the owner (RI-5), and the owner can successfully call each one (RI-6). • RI-18 <code>atMostOneCallerPassesOnlyOwner</code> ✓ passes: at any moment, only one address can call the twenty settings functions, and it is the owner. There can only be one owner at a time. FV — Setup is not a callable function after deploy: • RI-1 <code>setUpAlwaysRevertsAfterDeployment</code> ✓ passes: once the module list is set up, <code>setUp</code> always reverts, so nobody can re-run it to replace the owner or the module list.
G2.9	Only enabled modules of the Roles Modifier can execute transactions through it. Roles Modifier has no owner, since its owner value is <code>address(0)</code> .	FV — Only enabled modules can execute: • RI-7 <code>onlyEnabledModulesCanExec</code> ✓ passes: on all four execution entry points (<code>execTransactionFromModule</code>, <code>execTransactionFromModuleReturnData</code>, <code>execTransactionWithRole</code>, <code>execTransactionWithRoleReturnData</code>), a call that succeeds came from an address in the module list. No owner: • same as G4.RolesModifier.2.
G2.10	The Governor is the only enabled module of the Roles Modifier with an assignedRole.	On-chain — The Roles Modifier's module list contains only the Governor: • ✓ The Roles Modifier's (<code>0xaBAC51B6AEb05a2CE65310F79e64DF203D6c8Ab3</code>) module list returns just one, the Governor (<code>0x2B715634134220ffeEE9458b4e34E41A41418607</code>), and the list ends there (<code>getModulesPaginated</code>, block 26080168).

#	Statement	Evidence
		On-chain — The Roles Modifier's storage holds only the Governor as a module: •  Read directly from storage rather than through the Roles Modifier's own functions, the module list starts at the Governor and the Governor points back to the start, so it is the only entry (<code>modules</code>, storage slot 104, block 26080168). On-chain — The Governor holds a role: •  The Governor is recorded as a member of role 1 (<code>roles[1].members</code>, storage slot 107, block 26080168). On-chain — No other module has ever been added or given a role: •  The Roles Modifier's complete event history, from its creation at block 26042039 to block 26080168, has one <code>EnabledModule</code>, for the Governor, and no <code>DisabledModule</code>. It has one <code>AssignRoles</code>, also for the Governor.
G2.11	The Governor is assigned only role 1, its <code>defaultRole</code> .	On-chain — The Governor's default role is 1: •  <code>defaultRoles</code> on the Roles Modifier (<code>0xaBAC51B6AEb05a2CE65310F79e64DF203D6c8Ab3</code>) returns 1 for the Governor (<code>0x2B715634134220ffeEE9458b4e34E41A41418607</code>) at block 26080168. On-chain — The Governor is a member of role 1: •  The Governor is recorded as a member of role 1 (<code>roles[1].members</code>, storage slot 107, block 26080168). FV — A module's roles change only through <code>assignRoles</code>, and its default role only through <code>setDefaultRole</code>: • RI-19 <code>membershipOnlyChangesThroughAssignRoles</code>  passes: over every write function, <code>setUp</code> included, and any caller, a module's membership of a role changes only through <code>assignRoles</code> (<code>Roles.sol:290</code>), and every change emits <code>AssignRoles</code>. • RI-20 <code>defaultRoleOnlyChangesThroughSetDefaultRole</code>  passes: likewise, a module's default role changes only through <code>setDefaultRole</code> (<code>Roles.sol:310</code>), and every change emits <code>SetDefaultRole</code>. • RI-21 <code>ownerCanChangeMembershipThroughAssignRoles</code>  passes: the owner's <code>assignRoles</code> really does change a membership, so RI-19 is not satisfied by nothing working. • RI-4 <code>ownerCanStillReconfigure</code>  passes: the owner's <code>setDefaultRole</code> really does change a default role, so RI-20 is not satisfied by nothing working. On-chain — The Governor has only ever been given role 1: •  A module's roles can only change through <code>assignRoles</code> (<code>Roles.sol:290</code>), which always emits <code>AssignRoles</code> (RI-19). The Roles Modifier's complete event history, from its creation at block 26042039 to block 26080168, has exactly one <code>AssignRoles</code>: the Governor, role 1, member (tx <code>0x1d4ec5029b9c66cff2c92bc8be16196bba4d4484caaa74488bdcc61616fb186a</code>). It also has exactly one <code>SetDefaultRole</code> (RI-20): the Governor, role 1 (tx <code>0xf01e95980c86dc59667c78094ba1f7a65c744b5214029ef70e721ec8c340a928</code>).
G2.12	Role 1 and the <code>SetTxNonceGuard</code> together restrict the Governor to only vetoes.	FV — Role 1 alone restricts member module executions to vetoes (<code>Delay.setTxNonce(uint256)</code>), except for executions addressed to MultiSend: • RC-1 <code>roleConfigLimitsExecTransactionWithRoleToDelaySetTxNonce</code>,

#	Statement	Evidence
		RC-2 <code>roleConfigLimitsExecTransactionFromModuleToDelaySetTxNonce</code>, RC-3 <code>roleConfigLimitsExecTransactionWithRoleReturnDataToDelaySetTxNonce</code>, RC-4 <code>roleConfigLimitsExecTransactionFromModuleReturnDataToDelaySetTxNonce</code> ✓ pass: with no <code>SetTxNonceGuard</code> set on Roles Modifier, role 1's scope configuration admits only <code>setTxNonce(uint256)</code> on the Delay Modifier, with zero value and as a plain <code>Call</code>, on all four entry points, provided the destination is not the <code>MultiSend</code> address. • RC-5 <code>nonMemberExecTransactionWithRoleAlwaysReverts</code> ✓ passes: a caller that is not a member of the role it names can execute nothing at all. Stated on <code>execTransactionWithRole</code>. • RC-7 <code>withoutSetTxNonceGuardMultisendTargetEscapesRoleConfig</code> ✓ passes: without the guard, a transaction addressed to the <code>MultiSend</code> address can execute even though its target is not the Delay Modifier. This is the gap the guard closes (RS-6). FV — SetTxNonceGuard alone restricts all Roles Modifier executions to vetoes (Delay.setTxNonce(uint256)): • RS-1 <code>setTxNonceGuardLimitsExecTransactionWithRoleToDelaySetTxNonce</code>, RS-2 <code>setTxNonceGuardLimitsExecTransactionFromModuleToDelaySetTxNonce</code>, RS-3 <code>setTxNonceGuardLimitsExecTransactionWithRoleReturnDataToDelaySetTxNonce</code>, RS-4 <code>setTxNonceGuardLimitsExecTransactionFromModuleReturnDataToDelaySetTxNonce</code> ✓ pass: with <code>SetTxNonceGuard</code> installed, every execution on all four entry points is <code>setTxNonce(uint256)</code> on the Delay Modifier, with zero value and as a plain <code>Call</code>, for any caller, any role and any Roles configuration. • RS-5 <code>setTxNonceGuardLimitsToDelaySetTxNonceUnderMaximallyPermissiveRoles</code> ✓ passes: under the worst-case configuration, blanket <code>Clearance.Target</code> with <code>ExecutionOptions.Both</code>, every execution is <code>setTxNonce(uint256)</code> on the Delay Modifier, with zero value and as a plain <code>Call</code>. • RS-6 <code>setTxNonceGuardRejectsMultisendTarget</code> ✓ passes: a transaction addressed to the <code>MultiSend</code> address always reverts. • RS-7 <code>withoutSetTxNonceGuardPermissiveRolesAllowNonDelayCall</code> ✓ passes: with no guard and the same permissive configuration, a call to something other than the Delay Modifier succeeds, so it is the guard that imposes the restriction. FV — With role 1 and SetTxNonceGuard applied together, the Governor can only veto: • RG-2 <code>governorExecTransactionWithRoleLimitedToDelaySetTxNonce</code>, RG-3 <code>governorExecTransactionFromModuleLimitedToDelaySetTxNonce</code>, RG-4 <code>governorExecTransactionWithRoleReturnDataLimitedToDelaySetTxNonce</code>, RG-5 <code>governorExecTransactionFromModuleReturnDataLimitedToDelaySetTxNonce</code> ✓ pass: every call the Governor successfully executes through the Roles Modifier is <code>setTxNonce(uint256)</code> on the Delay Modifier, with zero value and as a plain <code>Call</code>, on all four execution entry points: <code>execTransactionWithRole</code>, <code>execTransactionFromModule</code>, <code>execTransactionWithRoleReturnData</code> and

#	Statement	Evidence
		<code>execTransactionFromModuleReturnData</code>. Assumes <code>SetTxNonceGuard</code> is installed and pointed at the Delay Modifier, and the Governor is an enabled module, a member of role 1 only, with default role 1 (G2.11). FV — The Governor successfully vetoes: • RG-6 <code>governorCanStillCallDelaySetTxNonce</code>  passes: with <code>SetTxNonceGuard</code> installed and pointed at the Delay Modifier, and the Governor enabled as a module and member of role 1 only with default role 1 (G2.11), the Governor's <code>setTxNonce</code> call really does go through, so the restriction is not achieved by nothing working. On-chain — The <code>SetTxNonceGuard</code> is the Roles Modifier's guard: •  <code>guard</code> on the Roles Modifier (<code>0xaBAC51B6AEb05a2CE65310F79e64DF203D6c8Ab3</code>) returns the <code>SetTxNonceGuard</code> (<code>0x7aE03372ECDcEe335CdEfB7d354d507F0506616C</code>), and <code>getGuard()</code> returns the same address (block 26095209).
G2.13	Only TERM token holders can create a Governor proposal to veto, and once the proposal passes, it can be executed successfully.	FV — Only TERM holders can propose: • GV-1 <code>holderAboveThresholdCanProposeVeto</code>  passes: any address holding at least 1,000 TERM of voting power at <code>clock() - 1</code> can propose the veto, the one-action proposal calling the Roles Modifier with <code>execTransactionWithRole(Delay, 0, setTxNonce(n), Call, 1, true)</code>, for any <code>n</code> and any description of up to 1,024 bytes accepted by <code>propose</code>, unless that exact proposal already exists. • GP-1 <code>callerBelowProposalThresholdCannotPropose</code>  passes: any caller whose TERM voting power at <code>clock() - 1</code> is below the proposal threshold—including callers with zero voting power, such as nonholders—cannot create proposals. FV — Only TERM holders can vote: • GC-1 <code>directVoteCountsOnlyTheVotersVotes</code>  passes: through <code>castVote</code>, <code>castVoteWithReason</code> and <code>castVoteWithReasonAndParams</code>, a proposal's tally grows by exactly the caller's votes at its snapshot if the caller voted on it, and does not move otherwise. • GC-2 <code>voteBySigCountsOnlyTheVotersVotes</code> and GC-3 <code>voteWithReasonAndParamsBySigCountsOnlyTheVotersVotes</code>  pass: through <code>castVoteBySig</code> and <code>castVoteWithReasonAndParamsBySig</code>, a proposal's tally grows by exactly the signing voter's votes at its snapshot if that voter voted on it, and does not move otherwise, whoever submits the call. • GC-4 <code>nothingButAVoteMovesATally</code>  passes: every other write function, <code>execute</code> and <code>relay</code> included, leaves every proposal tally where it was; <code>queue</code> never succeeds (GS-4). • GC-5 <code>termHolderVoteCounts</code>  passes: a caller with votes at the snapshot can vote, and the tally moves, so the rules are not satisfied by nothing working. On-chain — The Governor counts TERM votes, and its proposal threshold is 1,000 TERM: •  at block 26095237, <code>token()</code> on the Governor (<code>0x2B715634134220ffeEE9458b4e34E41A41418607</code>) returns the TERM token (<code>0xC3d21f79C3120A4fFda7A535f8005a7c297799bF</code>), and <code>proposalThreshold()</code> returns <code>10000000000000000000</code> (1,000 TERM). FV — Once it passes, the veto proposal executes and carries through to the Delay Modifier: • GV-2 <code>passedVetoReachesTheDelayOwnerSafe</code>  passes: once the veto's state is <code>Succeeded</code>, anyone's <code>execute</code> succeeds, and the Roles Modifier

#	Statement	Evidence
		forwards the veto to the DelayOwnerSafe as exactly one module call, <code>execTransactionFromModule(Delay, 0, setTxNonce(n), Call)</code>, for any <code>n</code>, provided that call returns true. Assumes the deployed wiring: <code>SetTxNonceGuard</code> is the Roles Modifier's guard and points at the Delay Modifier, role 1 is scoped to <code>Delay.setTxNonce</code>, and the Governor is an enabled module in role 1 (G2.10, G2.11). A recording stand-in takes the DelayOwnerSafe's place. • GV-3 <code>delayOwnerSafeLandsTheVeto</code>  passes: on GnosisSafe v1.4.1, an enabled module's <code>execTransactionFromModule(Delay, 0, setTxNonce(n), Call)</code> returns true and sets the Delay Modifier's <code>txNonce</code> to <code>n</code>, for any <code>n</code> the Delay Modifier accepts, when the Safe owns the Delay Modifier (G2.3).

Generalization 3 — The following: 1. the owners, fallbackHandler, modules, guard for the ProposerSafe, OwnerlessSafe, and DelayOwnerSafe, and 2. the owner, target, modules, guard, txCooldown, txExpiration for the Delay Modifier can be changed only by the Term multisig holders on the ProposerSafe, OwnerlessSafe, or DelayOwnerSafe.

ProposerSafe

Settings functions (GnosisSafe v1.3.0; only the Safe itself can call them):

- `enableModule(address)`
- `disableModule(address,address)`
- `addOwnerWithThreshold(address,uint256)`
- `removeOwner(address,address,uint256)`
- `swapOwner(address,address,address)`
- `changeThreshold(uint256)`
- `setGuard(address)`
- `setFallbackHandler(address)`
- `setup(address[],uint256,address,bytes,address,address,uint256,address)` , one-time initializer: always reverts once the Safe is set up

Conclusion: The modules, owners, guard, and fallbackHandler of the ProposerSafe cannot be changed other than by the Term multisig holders on the ProposerSafe.

ProposerSafe · Table 1 — The settings functions of the ProposerSafe can only be executed by its signers since it has no enabled modules.

#	Statement	Evidence
G3.ProposerSafe.1	Only Modules or Owners can interact with ProposerSafe, except <code>fallback</code> and <code>receive</code> , which anyone can call. The settings functions of the ProposerSafe can only be executed by Owners or enabled modules.	Same as G1.9.
G3.ProposerSafe.2	ProposerSafe fallback handler is <code>address(0)</code> .	Same as G1.10.
G3.ProposerSafe.3	ProposerSafe owners are Term multisig holders.	Same as G1.11.
G3.ProposerSafe.4	ProposerSafe has no enabled modules.	Same as G1.12.

OwnerlessSafe

Settings functions (GnosisSafe v1.3.0; only the Safe itself can call them):

- `enableModule(address)`
- `disableModule(address,address)`
- `addOwnerWithThreshold(address,uint256)`

- `removeOwner(address,address,uint256)`
- `swapOwner(address,address,address)`
- `changeThreshold(uint256)`
- `setGuard(address)`
- `setFallbackHandler(address)`
- `setup(address[],uint256,address,bytes,address,address,uint256,address)` , one-time initializer: always reverts once the Safe is set up

Conclusion: The modules, owners, guard, and fallbackHandler of the OwnerlessSafe cannot be changed other than by its signers or its only enabled module, the Delay Modifier. The Delay Modifier can only execute proposals queued by the signers of the ProposerSafe. Additional proposal modules can only be added by the signers of the DelayOwnerSafe.

OwnerlessSafe · Table 1 — Only OwnerlessSafe's owners and the Delay Modifier, the only enabled module, can modify the OwnerlessSafe's settings.

#	Statement	Evidence
G3.OwnerlessSafe.1	Only Modules or Owners can interact with OwnerlessSafe, except <code>fallback</code> and <code>receive</code> , which anyone can call. The settings functions of the OwnerlessSafe can only be executed by Owners or enabled modules.	Same as G1.2.
G3.OwnerlessSafe.2	OwnerlessSafe fallback handler is <code>address(0)</code> .	Same as G1.3.
G3.OwnerlessSafe.3	OwnerlessSafe owners are Term multisig holders.	Same as G1.4.
G3.OwnerlessSafe.4	Delay Modifier is the only enabled module on OwnerlessSafe.	Same as G1.5.
G3.OwnerlessSafe.5	Only Modules or Owner can interact with Delay Modifier, except <code>executeNextTx</code> and <code>skipExpired</code> , which anyone can call.	Same as G1.6.

After G3.OwnerlessSafe.5, the analysis branches at the Delay Modifier into its two sources of authority: its only enabled module, the ProposerSafe, which can propose transactions to the Delay Modifier; and its owner, the DelayOwnerSafe, which can modify the Delay Modifier's settings. Tables 2a and 2b establish the restrictions on each path.

OwnerlessSafe · Table 2a — The only enabled module of the Delay Modifier, the ProposerSafe, can propose transactions to the Delay Modifier queue. Each proposal cannot be executed until its cooldown has passed, expires if it is not executed in time, and can be vetoed.

#	Statement	Evidence
G3.OwnerlessSafe.6a	ProposerSafe is the only enabled module on the Delay Modifier.	Same as G1.8.
G3.OwnerlessSafe.7a	Only Modules or Owners can interact with ProposerSafe, except <code>fallback</code> and <code>receive</code> , which anyone can call.	Same as G1.9.

#	Statement	Evidence
G3.OwnerlessSafe. 8a	ProposerSafe fallback handler is <code>address(0)</code> .	Same as G1.10.
G3.OwnerlessSafe. 9a	ProposerSafe owners are Term multisig holders.	Same as G1.11.
G3.OwnerlessSafe. 10a	ProposerSafe has no enabled modules.	Same as G1.12.
G3.OwnerlessSafe. 11a	ProposerSafe can propose a transaction to the Delay Modifier that is executed on the OwnerlessSafe once the cooldown has passed, the expiration has not passed, the PauseGuard on the Delay Modifier is not paused, and the transaction has not been vetoed.	Same as G1.13.

OwnerlessSafe · Table 2b — The owner of the Delay Modifier, the DelayOwnerSafe, can call the Delay Modifier's settings functions. These functions can be executed only through the DelayOwnerSafe's signers, not through its only enabled module, the Roles Modifier.

#	Statement	Evidence
G3.OwnerlessSafe. 6b	Delay Modifier only has one owner, which is the DelayOwnerSafe.	Same as G2.3.
G3.OwnerlessSafe. 7b	Only Modules or Owners can interact with DelayOwnerSafe, except <code>fallback</code> and <code>receive</code> , which anyone can call.	Same as G2.4.
G3.OwnerlessSafe. 8b	DelayOwnerSafe fallback handler is <code>address(0)</code> .	Same as G2.5.
G3.OwnerlessSafe. 9b	DelayOwnerSafe owners are Term multisig holders.	Same as G2.6.
G3.OwnerlessSafe. 10b	DelayOwnerSafe only has one enabled module, the Roles Modifier.	Same as G2.7.
G3.OwnerlessSafe. 11b	Only Modules with assignedRoles or Owner can interact with Roles Modifier.	Same as G2.8.
G3.OwnerlessSafe. 12b	Roles Modifier has no owner, since its owner value is <code>address(0)</code> .	Same as G4.RolesModifier.2.
G3.OwnerlessSafe. 13b	The Governor is the only enabled module of the Roles Modifier with an assignedRole.	Same as G2.10.
G3.OwnerlessSafe. 14b	The Governor is assigned only role 1, its <code>defaultRole</code> .	Same as G2.11.

#	Statement	Evidence
G3.OwnerlessSafe.15b	Role 1 and the SetTxNonceGuard together restrict the Governor to only vetoes.	Same as G2.12.

DelayOwnerSafe

Settings functions (GnosisSafe v1.4.1; only the Safe itself can call them):

- `enableModule(address)`
- `disableModule(address,address)`
- `addOwnerWithThreshold(address,uint256)`
- `removeOwner(address,address,uint256)`
- `swapOwner(address,address,address)`
- `changeThreshold(uint256)`
- `setGuard(address)`
- `setFallbackHandler(address)`
- `setup(address[],uint256,address,bytes,address,address,uint256,address)` , one-time initializer: always reverts once the Safe is set up

Conclusion: The modules, owners, guard, and fallbackHandler of the DelayOwnerSafe cannot be changed other than by its signers. The DelayOwnerSafe's only enabled module, the Roles Modifier, cannot execute DelayOwnerSafe settings functions. The roles and members of the Roles Modifier cannot be changed by any caller. Additional modules can only be added to the DelayOwnerSafe by its own signers.

DelayOwnerSafe · Table 1 — The DelayOwnerSafe's settings can only be changed by its owners or through its only enabled module, the Roles Modifier.

#	Statement	Evidence
G3. DelayOwnerSafe.1	Only Modules or Owners can interact with DelayOwnerSafe, except fallback and receive, which anyone can call. The settings functions of the DelayOwnerSafe can only be executed by Owners or enabled modules.	Same as G2.4.
G3. DelayOwnerSafe.2	DelayOwnerSafe fallback handler is <code>address(0)</code> .	Same as G2.5.
G3. DelayOwnerSafe.3	DelayOwnerSafe owners are Term multisig holders.	Same as G2.6.
G3. DelayOwnerSafe.4	DelayOwnerSafe only has one enabled module, the Roles Modifier.	Same as G2.7.
G3. DelayOwnerSafe.5	Only Modules with assignedRoles or Owner can interact with Roles Modifier.	Same as G2.8.

After G3.DelayOwnerSafe.5, the analysis branches at the Roles Modifier, the DelayOwnerSafe's only enabled module, into its two sources of authority: its only enabled module, the Governor, which can execute only veto transactions; and its owner, which is `address(0)`, so no owner can execute or modify the Roles Modifier. Tables 2a and 2b establish the restrictions on each path.

DelayOwnerSafe · Table 2a — The Roles Modifier, the only enabled module of the DelayOwnerSafe, can only execute veto transactions (`Delay.setTxNonce(uint256)`) from the Governor contract.

#	Statement	Evidence
G3. DelayOwnerSafe.6a	The Governor is the only enabled module of the Roles Modifier with an assignedRole.	Same as G2.10.
G3. DelayOwnerSafe.7a	The Governor is assigned only role 1, its <code>defaultRole</code> .	Same as G2.11.
G3. DelayOwnerSafe.8a	Role 1 and the <code>SetTxNonceGuard</code> together restrict the Governor to only vetoes.	Same as G2.12.

DelayOwnerSafe · Table 2b — The Roles Modifier, the only enabled module of the DelayOwnerSafe, has no owner, since its owner value is `address(0)`. The Roles Modifier's settings cannot be changed by anyone.

#	Statement	Evidence
G3. DelayOwnerSafe.6b	Roles Modifier has no owner, since its owner value is <code>address(0)</code> .	Same as G4.RolesModifier.2.

Delay Modifier

Settings functions (zodiac-modifier-delay v1.0.1; `onlyOwner`):

- `setTxCooldown(uint256)`
- `setTxExpiration(uint256)`
- `setAvatar(address)`
- `setTarget(address)`
- `enableModule(address)`
- `disableModule(address,address)`
- `setGuard(address)`
- `transferOwnership(address)`
- `renounceOwnership()`
- `setUp(bytes)`, one-time initializer: always reverts after deployment

`setTxNonce(uint256)`, the veto, is also `onlyOwner` but is excluded from this list because it does not make any of the changes covered by the conclusion below.

Conclusion: The modules, owner, guard, avatar, target, txCooldown, and txExpiration of the Delay Modifier cannot be changed other than by its owner, the DelayOwnerSafe. Only the signers of the DelayOwnerSafe can execute Delay Modifier settings functions. The DelayOwnerSafe's only enabled module, the Roles Modifier, cannot execute DelayOwnerSafe settings functions. The roles and members of the Roles Modifier cannot be changed by any caller. Additional modules can only be added to the DelayOwnerSafe by its own signers.

Delay Modifier · Table 1 — Only the owner of the Delay Modifier, the DelayOwnerSafe, can call the Delay Modifier's settings functions. The DelayOwnerSafe can only receive interactions from its owners or from its only enabled module, the Roles Modifier.

#	Statement	Evidence
G3.DelayModifier.1	Only Modules or Owner can interact with Delay Modifier, except <code>executeNextTx</code> and <code>skipExpired</code> , which anyone can call. The settings functions are owner-only: only the Owner can successfully execute them.	Same as G1.6.
G3.DelayModifier.2	Delay Modifier only has one owner, which is the DelayOwnerSafe.	Same as G2.3.
G3.DelayModifier.3	Only Modules or Owners can interact with DelayOwnerSafe, except <code>fallback</code> and <code>receive</code> , which anyone can call.	Same as G2.4.
G3.DelayModifier.4	DelayOwnerSafe fallback handler is <code>address(0)</code> .	Same as G2.5.
G3.DelayModifier.5	DelayOwnerSafe owners are Term multisig holders.	Same as G2.6.
G3.DelayModifier.6	DelayOwnerSafe only has one enabled module, the Roles Modifier.	Same as G2.7.
G3.DelayModifier.7	Only Modules with assignedRoles or Owner can interact with Roles Modifier.	Same as G2.8.

After G3.DelayModifier.7, the analysis branches at the Roles Modifier, the DelayOwnerSafe's only enabled module, into its two sources of authority: its only enabled module, the Governor, which can execute only veto transactions; and its owner, which is `address(0)`, so no owner can execute or modify the Roles Modifier. Tables 2a and 2b establish the restrictions on each path.

Delay Modifier · Table 2a — The Roles Modifier, the only enabled module of the DelayOwnerSafe, can only execute veto transactions (`Delay.setTxNonce(uint256)`) from the Governor contract.

#	Statement	Evidence
G3.DelayModifier.8a	The Governor is the only enabled module of the Roles Modifier with an assignedRole.	Same as G2.10.
G3.DelayModifier.9a	The Governor is assigned only role 1, its <code>defaultRole</code> .	Same as G2.11.
G3.DelayModifier.10a	Role 1 and the <code>SetTxNonceGuard</code> together restrict the Governor to only vetoes.	Same as G2.12.

Delay Modifier · Table 2b — The Roles Modifier, the only enabled module of the DelayOwnerSafe, has no owner, since its owner value is `address(0)`. The Roles Modifier's settings cannot be changed by anyone.

#	Statement	Evidence
G3.DelayModifier.8b	Roles Modifier has no owner, since its owner value is <code>address(0)</code> .	Same as G4.RolesModifier.2.

Generalization 4 — The following: 1. the owner, target, modules, guard, role targets, role scopes, role assignments for the Roles Modifier, and 2. the voting token, proposal threshold, voting threshold, quorum for the Governor cannot be changed by anyone.

Roles Modifier

Settings functions (Roles v1.0.0; `onlyOwner`):

- `setMultisend(address)`
- `allowTarget(uint16,address,ExecutionOptions)`
- `revokeTarget(uint16,address)`
- `scopeTarget(uint16,address)`
- `scopeAllowFunction(uint16,address,bytes4,ExecutionOptions)`
- `scopeRevokeFunction(uint16,address,bytes4)`
- `scopeFunction(uint16,address,bytes4,bool[],ParameterType[],Comparison[],bytes[],ExecutionOptions)`
- `scopeFunctionExecutionOptions(uint16,address,bytes4,ExecutionOptions)`
- `scopeParameter(uint16,address,bytes4,uint256,ParameterType,Comparison,bytes)`
- `scopeParameterAsOneOf(uint16,address,bytes4,uint256,ParameterType,bytes[])`
- `unscopeParameter(uint16,address,bytes4,uint8)`
- `assignRoles(address,uint16[],bool[])`
- `setDefaultRole(address,uint16)`
- `setAvatar(address)`
- `setTarget(address)`
- `enableModule(address)`
- `disableModule(address,address)`
- `setGuard(address)`
- `transferOwnership(address)`
- `renounceOwnership()`
- `setUp(bytes)`, one-time initializer: always reverts after deployment

Conclusion: The owner, target, modules, guard, role targets, role scopes, and role assignments on the Roles Modifier cannot be changed by any caller.

Roles Modifier · Table 1 — Only the Roles Modifier's owner can execute settings functions. The Roles Modifier has no owner, since its owner value is `address(0)`. The Roles Modifier's settings cannot be changed by anyone.

#	Statement	Evidence
G4.RolesModifier.1	Only Modules with assignedRoles or Owner can interact with Roles Modifier. The settings functions are owner-only: only the Owner can successfully execute them.	Same as G2.8.

#	Statement	Evidence
G4.RolesModifier.2	Roles Modifier has no owner, since its owner value is <code>address(0)</code> .	FV — Settings functions are unreachable when owner is <code>address(0)</code>: • RI-15 <code>noOwnerSettingsAlwaysRevert</code>  passes: with the owner at <code>address(0)</code>, each of the twenty settings functions reverts for every caller, because no user can submit a transaction from <code>address(0)</code>, so the owner-only check can never be satisfied. • RI-16 <code>noOwnerStaysNoOwner</code>  passes: no function can set the Roles Modifier's owner to a nonzero address (<code>setUp</code> is covered by RI-1, G2.8). • RI-17 <code>renounceOwnershipLeavesNoOwner</code>  passes: the owner's <code>renounceOwnership()</code> succeeds and leaves the Roles Modifier with no owner, so RI-15 and RI-16 are not about an unreachable state. On-chain — Owner is <code>address(0)</code>: •  pending. <code>owner()</code> on the Roles Modifier (<code>0xaBAC51B6AEb05a2CE65310F79e64DF203D6c8Ab3</code>) must return <code>0x00</code>, which the <code>OwnerlessSafe</code> sets by calling <code>renounceOwnership()</code>. At block 26083996 it returns the <code>OwnerlessSafe</code> (<code>0xb8A1dF43c1c88b13937C0c5CEBbAd15830cAeC03</code>), and <code>_owner</code> (slot 51, <code>OwnableUpgradeable</code>) holds the same address.

Governor

Settings functions: none.

Conclusion: The voting token, proposal threshold, voting threshold, and quorum cannot be modified by anyone.

Governor · Table 1 — Nobody can change the Governor's settings.

#	Statement	Evidence
G4.Governor.1	Governor does not have settings functions.	FV — Every write function of the Governor is enumerated below: GS-1 <code>governorWriteFunctionsAreTheKnownFourteen</code>  passes: <code>TermFinanceGovernor</code> builds on <code>OpenZeppelin's Governor</code>, <code>GovernorVotes</code> and <code>GovernorCountingSimple</code>. It has no write functions besides these fourteen: <i>Proposal lifecycle:</i> • <code>propose</code> • <code>execute</code> • <code>cancel</code> <i>Voting:</i> • <code>castVote</code> • <code>castVoteWithReason</code> • <code>castVoteWithReasonAndParams</code> • <code>castVoteBySig</code> • <code>castVoteWithReasonAndParamsBySig</code> <i>Governance only — callable only by the Governor itself, as an action of an executed proposal:</i> • <code>relay</code>, which makes one call out and writes nothing <i>Token and ETH receipt — each writes nothing:</i> • <code>onERC721Received</code> • <code>onERC1155Received</code>

#	Statement	Evidence
		• onERC1155BatchReceived • receive <i>Non-interactions — this always reverts, so none of its effects persist:</i> • queue FV — No function changes a setting: • GS-2 governorSettingsNeverChange  passes: for every write function except queue (GS-4), any caller and any arguments, every setting reads the same afterwards: name, version, COUNTING_MODE, the executor, token, votingDelay, votingPeriod, proposalThreshold, quorumNumerator, quorumDenominator and proposalNeedsQueuing. FV — queue always reverts: • GS-4 queueAlwaysReverts  passes: the Governor has no timelock, so it never overrides OpenZeppelin's _queueOperations, which returns 0, and queue then reverts with GovernorQueueNotImplemented. A proposal that passes goes straight to execute.